

**Федеральное государственное автономное образовательное
учреждение высшего образования
Национальный исследовательский университет ИТМО**



Факультет Программной Инженерии и Компьютерной Техники

**Отчёт по проекту
«Тестер реакции»
по дисциплине**

«Сетевые технологии интернета вещей»

Выполнили:

**Ястребов-Амирханов Алекси
Исупов Никита Александрович
Ваганова Мария Александровна**

Группа:

P3332

Преподаватель:

Гончаров Алексей Андреевич

Санкт-Петербург,

2026

Содержание

Постановка задачи и описание проекта	3
Общая архитектура системы.....	4
Аппаратная часть	5
Программная архитектура.....	6
Функциональные возможности системы.....	8
Пользовательский интерфейс	9
Игровая логика	9
Работа с сетью (Wi-Fi).....	10
Взаимодействие с сервером и веб-интерфейсы	11
Эксплуатация и оптимизация	12
Ограничения и развитие.....	13
Заключение и вывод.....	14

Постановка задачи и описание проекта

Цель проекта

Разработать аппаратно-программный тренажер реакции, который позволяет игроку проходить серию раундов на физическом устройстве, измерять время реакции и сохранять результаты на сервере. Система объединяет микроконтроллерное устройство на базе NodeMCU ESP8266 и backend-приложение на Spring Boot.

Основная задача тренажера состоит в том, чтобы после случайной задержки показать игроку целевую кнопку, зафиксировать нажатие и определить результат раунда. В зависимости от поведения игрока раунд может завершиться успешным нажатием, ошибкой выбора кнопки, пропуском или фальстартом. По итогам игры сервер рассчитывает среднее время реакции, лучший результат, количество промахов, неправильных нажатий и фальстартов.

Проект решает несколько прикладных задач:

1. регистрация игроков и авторизация через JWT;
2. регистрация физических устройств и выдача device token;
3. сопряжение игрока и устройства через шестизначный код;
4. обмен сообщениями между устройством и сервером через WebSocket;
5. проведение игровой сессии из заданного количества раундов;
6. сохранение истории сессий, результатов и статистики в PostgreSQL;
7. предоставление REST API для управления игроками, устройствами, сессиями и лидербордом.

В результате получается система, которую можно использовать как учебный пример IoT-проекта: есть аппаратная часть, прошивка, сетевой протокол, backend, база данных и API-документация.

Ссылка на проект:

<https://github.com/nifreebie/reaction-trainer>

Общая архитектура системы

Система построена по клиент-серверной архитектуре. В роли аппаратного клиента выступает плата NodeMCU ESP8266 с подключенными устройствами ввода и вывода. Серверная часть реализована на Java 17 и Spring Boot 4. Для хранения данных используется PostgreSQL. В проекте также присутствует OpenAPI-описание REST API и Docker Compose-конфигурация для локального запуска базы данных.

Основные компоненты системы:

1. аппаратное устройство: NodeMCU ESP8266, клавиатура для ввода кода, панель с кнопками и светодиодами, дисплей, зуммер;
2. прошивка: Arduino/C++ код, который управляет вводом, индикацией, звуком, Wi-Fi и WebSocket-соединением;
3. backend: Spring Boot-приложение, реализующее REST API, WebSocket endpoint, бизнес-логику игры и работу с БД;
4. база данных: PostgreSQL для хранения игроков, устройств, кодов сопряжения, сессий, раундов и результатов;
5. API-клиент или веб-интерфейс: внешний пользовательский слой, который работает с REST API для регистрации, входа, создания pair-code и просмотра результатов.

Упрощенный поток работы выглядит так:

1. Игрок регистрируется или входит в систему через REST API.
2. Физическое устройство регистрируется на сервере и получает токен.
3. Устройство подключается к Wi-Fi и открывает WebSocket-соединение с сервером.
4. Игрок создает код сопряжения.
5. Код вводится на клавиатуре устройства.
6. Сервер связывает игрока и устройство, создает игровую сессию и запускает игру.
7. Устройство получает команды раундов, показывает стимулы и отправляет результаты.

- Backend сохраняет данные, считает статистику и отправляет сообщение о завершении игры.

Аппаратная часть

Аппаратная часть проекта рассчитана на NodeMCU v3 Lolin на базе ESP8266.

Используемые компоненты:

- NodeMCU v3 Lolin / ESP8266;
- клавиатура 4x3 для ввода шестизначного кода сопряжения;
- I2C-расширитель PCF8574 для подключения клавиатуры;
- TM1638-подобная панель с 8 кнопками, 8 LED и 7-сегментным дисплеем;
- buzzer для коротких звуковых сигналов и финальной мелодии;
- питание для платы и периферийных модулей.

Назначение пинов в прошивке:

Назначение	NodeMCU pin	GPIO
I2C SDA	D2	GPIO4
I2C SCL	D3	GPIO0
Buzzer	D1	GPIO5
TM STB	D5	GPIO14
TM CLK	D6	GPIO12
TM DIO	D7	GPIO13

Клавиатура подключается через PCF8574 по I2C. По умолчанию используется адрес 0x20. Раскладка клавиатуры:

1	2	3
4	5	6
7	8	9

*	0	#
---	---	---

Клавиша * очищает введенный код, а клавиша # отправляет код на backend. Основная игровая панель используется уже во время раундов: сервер отправляет номер целевой кнопки от 1 до 8, устройство включает соответствующий LED и ожидает нажатие.

Программная архитектура

Backend реализован как Spring Boot-приложение с классическим разделением на слои:

1. controller - REST-контроллеры для авторизации, игроков, устройств, кодов сопряжения, сессий и результатов;
2. service и service.impl - бизнес-логика приложения;
3. repository - Spring Data JPA-репозитории для доступа к данным;
4. domain - JPA-сущности предметной области;
5. dto - объекты запросов, ответов и WebSocket-сообщений;
6. config - настройки безопасности, WebSocket, CORS, JSON и игровых параметров;
7. util - вспомогательные классы, JWT-фильтр, обработчик WebSocket и исключения.

Ключевые доменные сущности:

1. Player - пользователь системы;
2. Device - зарегистрированное физическое устройство;
3. PairCode - временный код сопряжения игрока и устройства;
4. GameSession - игровая сессия;
5. ReactionRound - отдельный раунд реакции;
6. Result - сохраненный результат игрока.

Для авторизации игроков используется Spring Security и JWT. Пароли хранятся в зашифрованном виде с помощью BCrypt. Большинство REST endpoint защищены JWT, кроме регистрации/входа и регистрации устройства. Устройство подключается к WebSocket endpoint /ws/devices с параметрами deviceId и token; проверка выполняется на этапе handshake.

Игровые настройки вынесены в application.yaml:

```
app:
  game:
    rounds-count: 10
    timeout-ms: 1500
    target-buttons-count: 8
    stimulus-delay-min-ms: 500
    stimulus-delay-max-ms: 2000
```

За игровой процесс отвечает GameFlowServiceImpl. Сервис запускает сессию, создает раунды, выбирает случайную целевую кнопку, отправляет команды устройству, принимает результаты и обновляет статистику. Сопряжение реализовано в PairingServiceImpl: сервер проверяет код, online-статус устройства, связывает устройство с игроком и создает сессию в статусе WAITING.

Прошивка написана на Arduino/C++ и содержит собственную машину состояний:

1. ENTER_PAIR_CODE - ввод кода сопряжения;
2. WAIT_PAIRING - ожидание ответа сервера;
3. COUNTDOWN - обратный отсчет перед игрой;
4. WAIT_ROUND - ожидание следующего раунда;
5. ROUND_PAUSE - пауза между раундами;
6. WAIT_STIMULUS - ожидание появления стимула;
7. WAIT_PRESS - ожидание нажатия кнопки;

8. GAME_FINISHED - отображение результата и ожидание сброса.

Функциональные возможности системы

Система предоставляет полный цикл работы с тренажером реакции: от регистрации игрока и устройства до прохождения игры и анализа результатов.

Основные возможности:

1. регистрация и вход игрока;
2. получение JWT-токена для защищенных API-запросов;
3. регистрация устройства с выдачей device token;
4. подключение устройства к серверу по WebSocket;
5. создание и проверка активного pair-code;
6. ввод кода на физической клавиатуре;
7. автоматический старт игровой сессии после успешного сопряжения;
8. проведение игры из 10 раундов по умолчанию;
9. случайный выбор целевой кнопки и задержки стимула;
10. обработка успешных нажатий, ошибок, пропусков и фальстартов;
11. расчет среднего и лучшего времени реакции;
12. хранение истории сессий и результатов;
13. просмотр статистики игрока и лидерборда.

Пользовательский интерфейс

В текущем проекте пользовательский интерфейс представлен двумя уровнями: физическим интерфейсом устройства и программным REST API, на основе которого может быть построен веб-интерфейс.

Физический интерфейс включает:

1. клавиатуру 4x3 для ввода кода сопряжения;
2. 7-сегментный дисплей для отображения введенного кода, обратного отсчета, номера целевой кнопки и итогового результата;
3. LED-индикацию целевой кнопки на игровой панели;
4. кнопки игровой панели для реакции игрока;
5. звуковые сигналы buzzer для обратной связи.

С точки зрения игрока сценарий прост: получить код в клиентском интерфейсе, ввести его на устройстве, дождаться обратного отсчета, нажимать подсвеченные кнопки и после завершения увидеть итоговое среднее время реакции.

REST API выполняет роль интерфейса для веб-клиента или мобильного приложения. Через него можно регистрироваться, входить в систему, создавать коды сопряжения, смотреть активную сессию, историю игр, статистику и таблицу лидеров.

Игровая логика

Игровая логика строится вокруг сессии и раундов. После успешного сопряжения backend создает `GameSession`, переводит ее в активный статус и отправляет устройству первый `round_start`.

Для каждого раунда сервер:

1. выбирает случайную целевую кнопку в диапазоне от 1 до 8;
2. выбирает случайную задержку появления стимула;
3. сохраняет раунд в базе данных;
4. отправляет устройству WebSocket-сообщение с номером раунда, целевой кнопкой, задержкой и timeout.

Устройство после получения раунда показывает стимул, включает LED и измеряет время от появления стимула до нажатия.

Возможные результаты:

1. HIT - игрок нажал правильную кнопку вовремя;
2. WRONG_BUTTON - игрок нажал неправильную кнопку;
3. MISS - игрок не нажал кнопку за отведенное время;
4. FALSE_START - игрок нажал кнопку до появления стимула.

Backend дополнительно нормализует результат. Если устройство отправило некорректные данные, сервер проверяет timeout, номер целевой кнопки и наличие времени реакции. Это повышает надежность системы, так как итоговый результат не зависит только от прошивки.

После последнего раунда сессия переводится в статус FINISHED, сохраняется лучший результат игрока, а на устройство отправляется сообщение game_finished.

Работа с сетью (Wi-Fi)

Wi-Fi используется для подключения NodeMCU ESP8266 к локальной сети, в которой доступен backend. В прошивке задаются:

```
const char* WIFI_SSID = "YOUR_WIFI";
const char* WIFI_PASSWORD = "YOUR_WIFI_PASSWORD";
const char* BACKEND_HOST = "192.168.1.10";
const uint16_t BACKEND_PORT = 8080;
```

После включения устройство работает в режиме WIFI_STA, подключается к заданной сети и выводит IP-адрес в Serial Monitor. Затем прошивка открывает WebSocket-соединение:

```
ws://BACKEND_HOST:8080/ws/devices?deviceId=nodemcu-01&token=DEVICE_TOKEN
```

Для поддержания online-статуса устройство отправляет heartbeat каждые 5 секунд. На сервере настроен timeout heartbeat: если сообщения долго не приходят, устройство считается offline. По умолчанию timeout равен 20 секундам, а проверка offline-статуса выполняется периодически.

При разрыве WebSocket-соединения прошивка возвращается к состоянию ввода кода, а библиотека WebSockets пытается переподключиться с интервалом 3 секунды.

Взаимодействие с сервером и веб-интерфейсы

Взаимодействие с сервером делится на REST API и WebSocket-протокол. REST API используется игроком или веб-интерфейсом.

Основные endpoint:

Метод	Endpoint	Назначение
POST	/api/auth/register	регистрация игрока
POST	/api/auth/login	вход игрока
GET	/api/me	получение данных текущего игрока
POST	/api/devices/register	регистрация устройства
GET	/api/devices/me	список устройств игрока
POST	/api/pair-codes	создание кода сопряжения
GET	/api/pair-codes/active	активный код сопряжения
GET	/api/sessions/current	текущая сессия
GET	/api/sessions/me	история сессий
GET	/api/sessions/{id}/rounds	раунды сессии
GET	/api/results/leaders	таблица лидеров
GET	/api/results/me/best	лучший результат игрока

WebSocket используется физическим устройством.

Сообщения от устройства:

```
{ "type": "heartbeat" }
{ "type": "pair_request", "code": "123456" }
{ "type": "round_result", "sessionId": "uuid", "roundNumber": 1, "pressedButton": 2, "reactionTimeMs": 350, "result": "HIT" }
```

Сообщения от backend:

```
{ "type": "connected", "deviceId": "nodemcu-01" }
{ "type": "pair_success", "sessionId": "uuid", "playerName": "player1" }
{ "type": "round_start", "sessionId": "uuid", "roundNumber": 1, "targetButton": 2, "stimulusDelayMs": 800, "timeoutMs": 1500 }
{ "type": "game_finished", "sessionId": "uuid", "avgReactionMs": 360, "bestReactionMs": 280, "missesCount": 1, "wrongButtonsCount": 0, "falseStartsCount": 0 }
{ "type": "error", "message": "Invalid or expired code" }
```

Эксплуатация и оптимизация

Для локальной эксплуатации требуются JDK 17, Maven, Docker и Docker Compose. База данных PostgreSQL запускается через compose.yaml. Backend читает параметры подключения из .env:

```
POSTGRES_USER=nifreebie
POSTGRES_PASSWORD=nis150905
POSTGRES_URL=jdbc:postgresql://localhost:5432/reaction_trainer_db
```

Типичный запуск:

```
docker compose up -d
mvn spring-boot:run
```

После запуска backend доступен на <http://localhost:8080>. Прошивку необходимо открыть в Arduino IDE, указать Wi-Fi, IP backend, DEVICE_ID и DEVICE_TOKEN, затем загрузить на NodeMCU.

Оптимизация в проекте уже частично заложена:

1. коды сопряжения заранее генерируются в пуле, что снижает задержку при запросе нового кода;
2. устаревшие и использованные коды периодически удаляются;
3. online-статус устройства контролируется через heartbeat;
4. WebSocket-сессии устройств хранятся в отдельном registry;

5. игровая логика на сервере проверяет и нормализует результаты;
6. настройки игры вынесены в конфигурационный файл без необходимости менять код.

Тестовый профиль использует H2 in-memory database, что упрощает запуск тестов без внешней PostgreSQL.

Ограничения и развитие

Текущая версия проекта решает основную задачу, но имеет ряд ограничений:

1. веб-интерфейс как отдельное frontend-приложение в репозитории отсутствует, взаимодействие выполняется через REST API;
2. прошивка рассчитана на конкретную схему подключения и конкретные модули;
3. WebSocket endpoint разрешает все origins, что удобно для разработки, но требует уточнения для production;
4. секрет JWT в application.yaml задан статически и должен быть вынесен в переменные окружения для реального развертывания;
5. device token отображается только при регистрации устройства, поэтому при его потере требуется повторная регистрация или механизм перевыпуска;
6. игровой режим пока фактически один - CLASSIC;
7. точность измерения зависит от аппаратной реализации, качества кнопок, задержек Wi-Fi и обработки цикла прошивки.

Возможные направления развития:

1. разработать полноценный веб-интерфейс для игроков и администратора;
2. добавить разные режимы игры: тренировка точности, быстрый старт, дуэль, марафон;
3. добавить настройку количества раундов и timeout для конкретной сессии;

4. реализовать OTA-обновление прошивки;
5. добавить графики прогресса игрока;
6. добавить управление несколькими устройствами и комнатами;
7. улучшить безопасность WebSocket и хранение секретов;
8. добавить интеграционные тесты WebSocket-протокола;
9. сделать корпус устройства и схему подключения в виде отдельной документации.

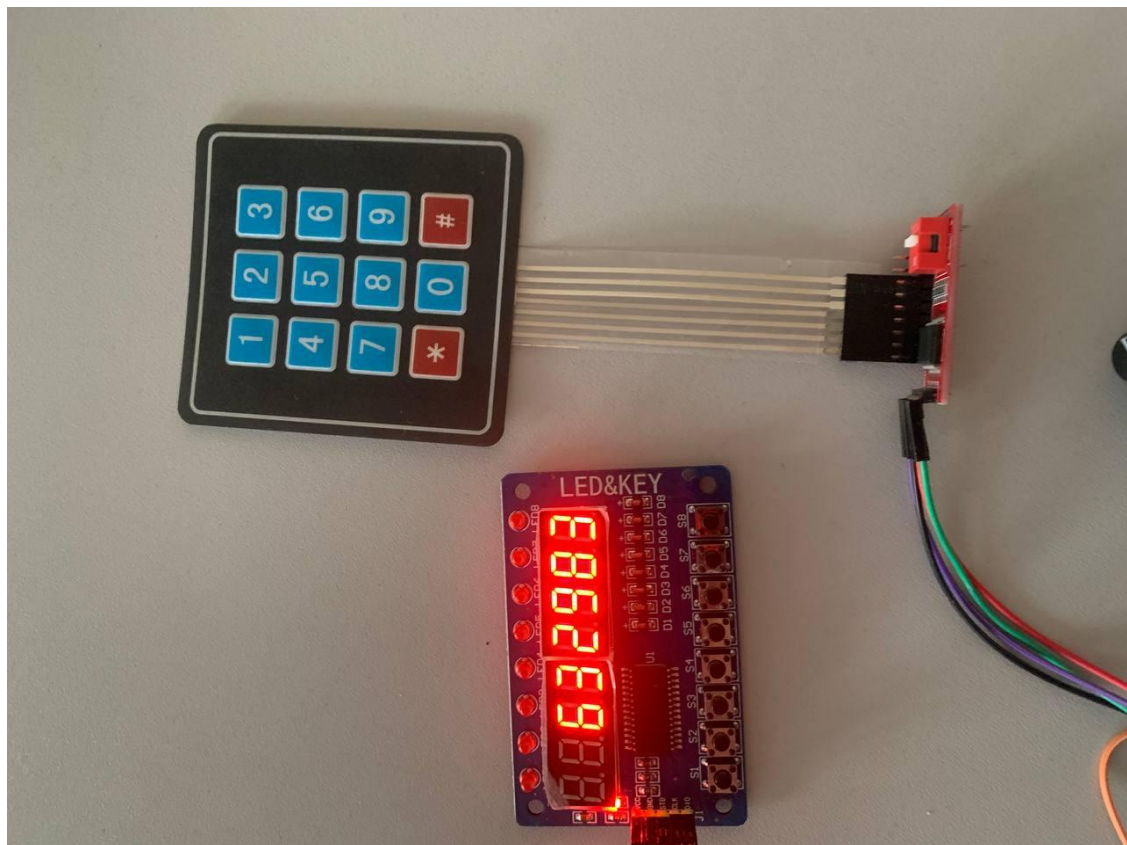
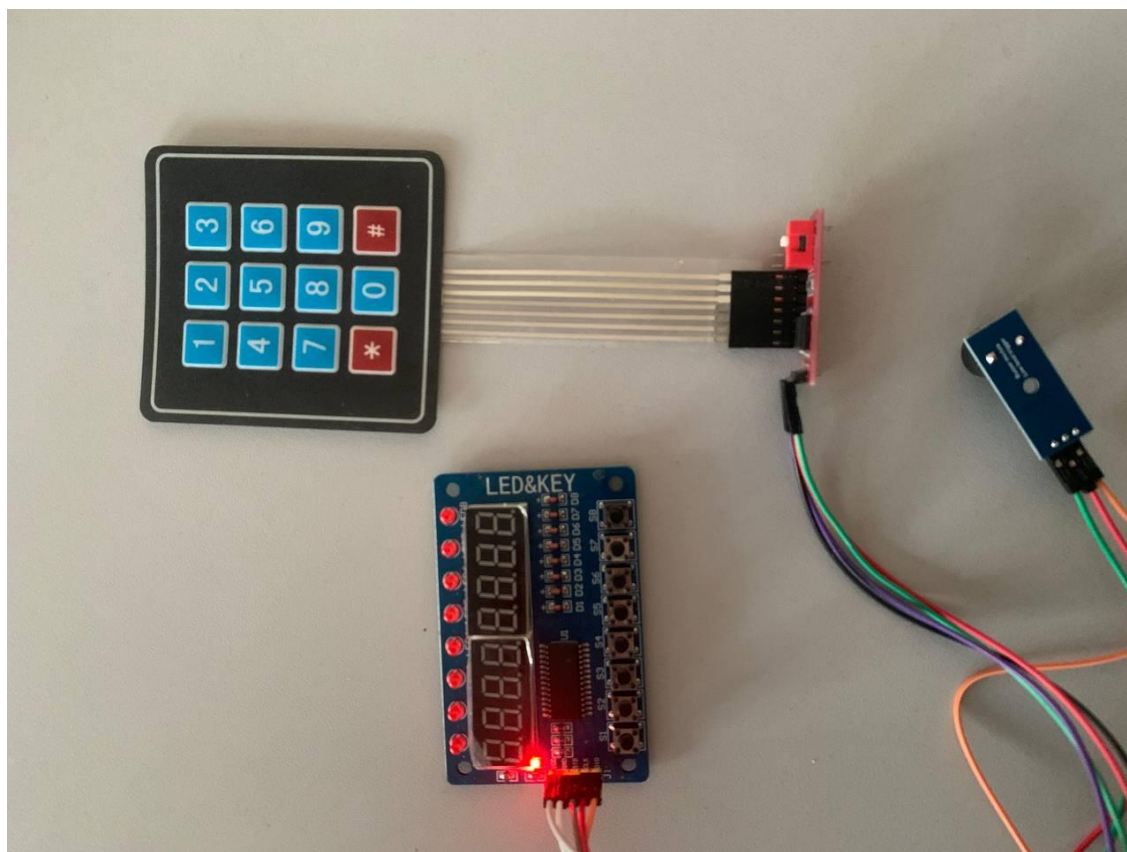
Заключение и вывод

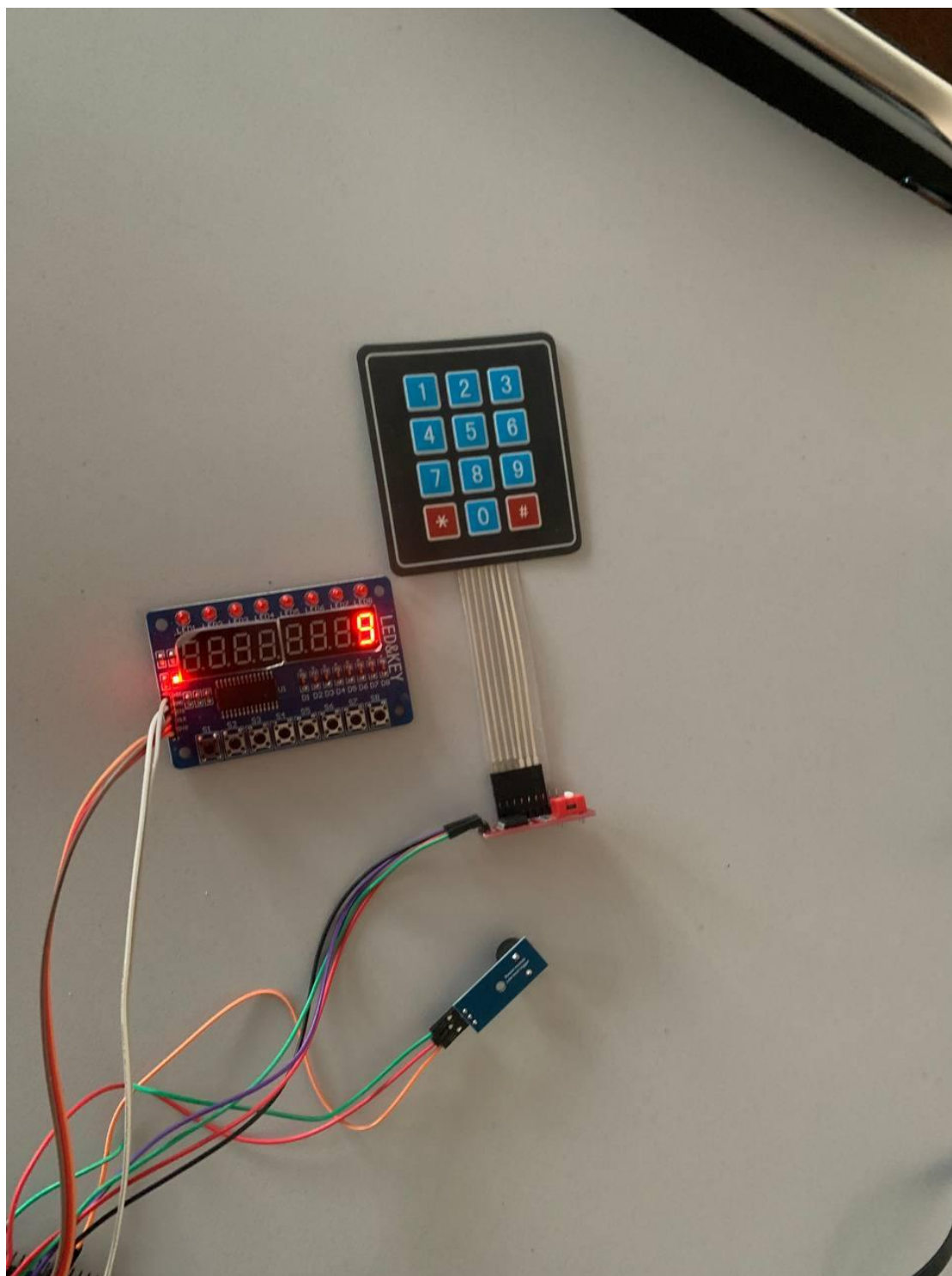
Проект Reaction Trainer представляет собой завершенный пример аппаратно-программной системы для тренировки реакции. В нем объединены физическое устройство на ESP8266, прошивка на Arduino/C++, сервер на Spring Boot, база данных PostgreSQL, JWT-авторизация, REST API и WebSocket-протокол для связи с устройством.

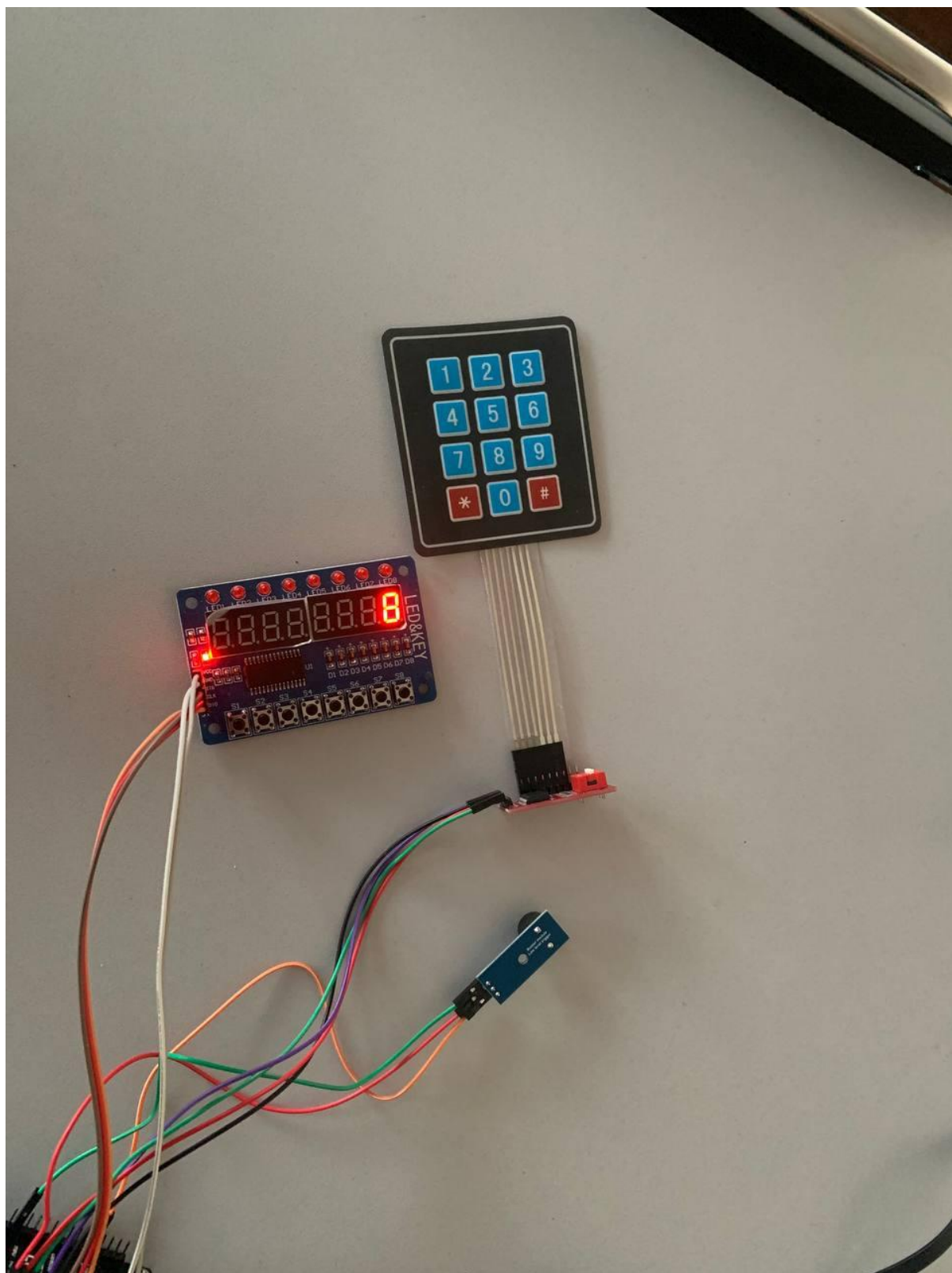
Главное достоинство проекта - понятная архитектура и полный пользовательский сценарий: игрок регистрируется, получает код, вводит его на устройстве, проходит игру, а результаты сохраняются и доступны через API. Серверная часть не только принимает данные от устройства, но и проверяет корректность результата, считает статистику и управляет игровым процессом.

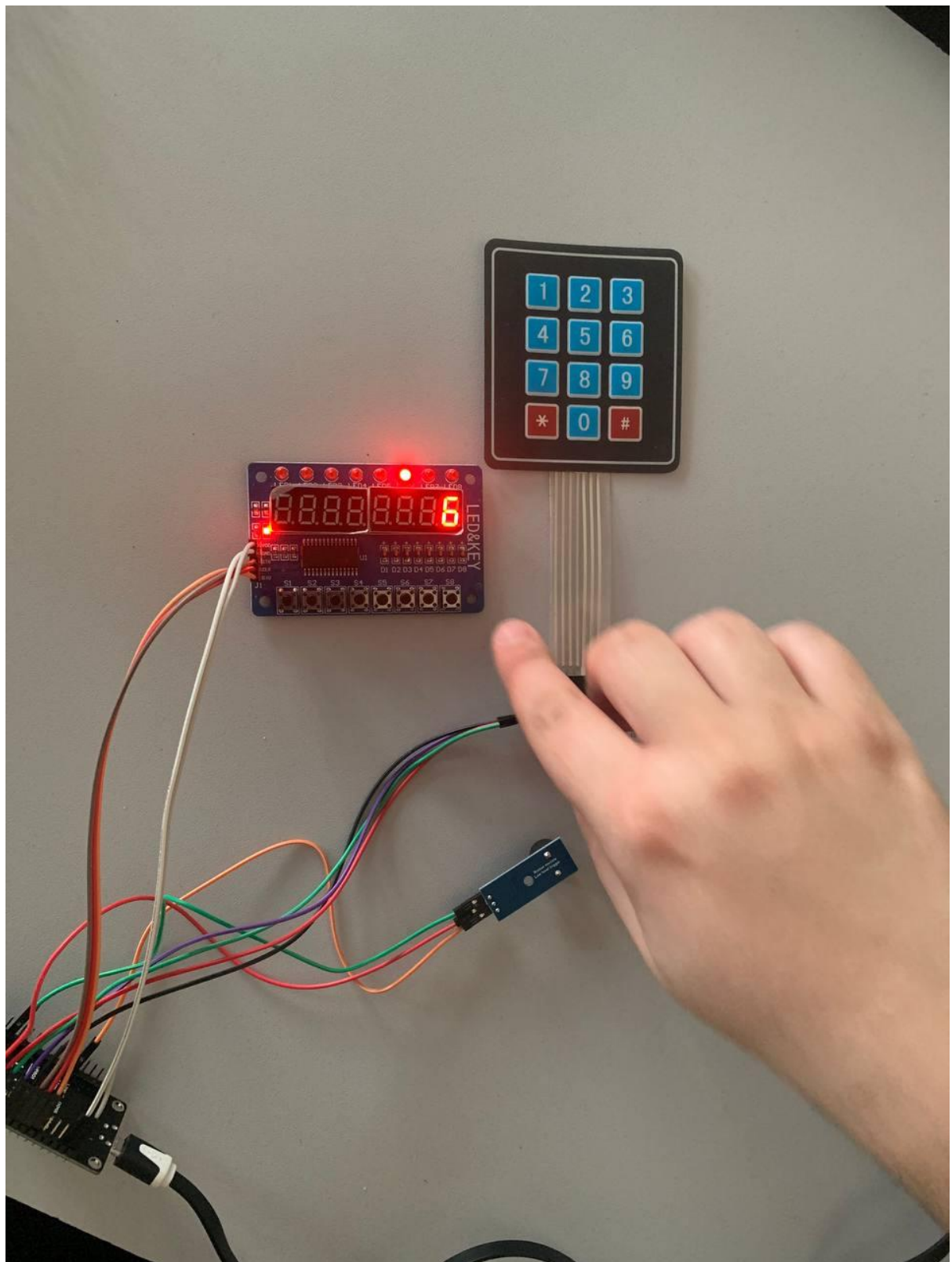
В дальнейшем проект можно развивать в сторону полноценного продукта: добавить frontend, расширить игровые режимы, повысить безопасность, улучшить аналитику и подготовить устройство к более удобной эксплуатации. Уже в текущем виде система демонстрирует основные принципы IoT-разработки и может использоваться как учебный или демонстрационный проект.

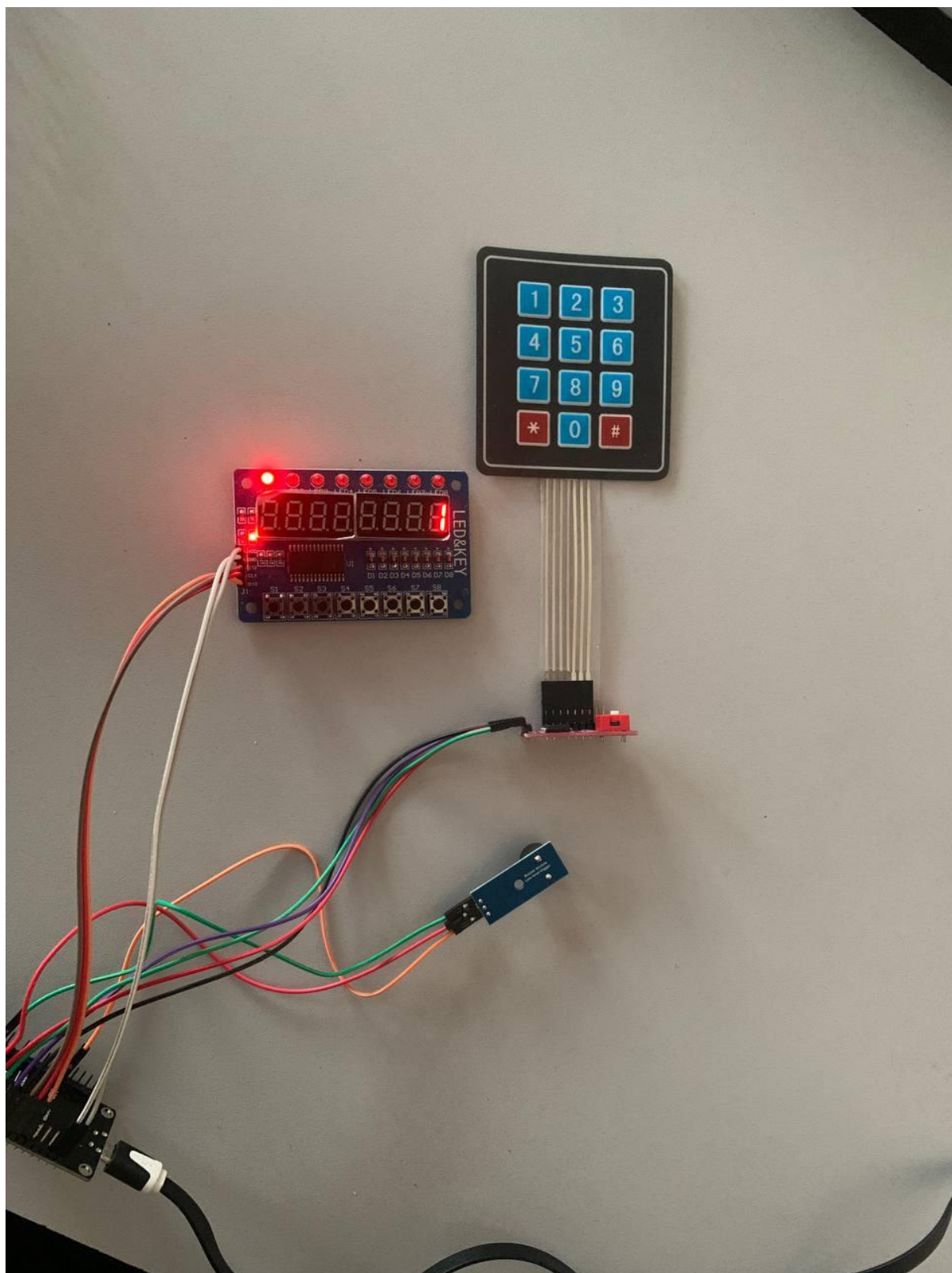
Приложение:

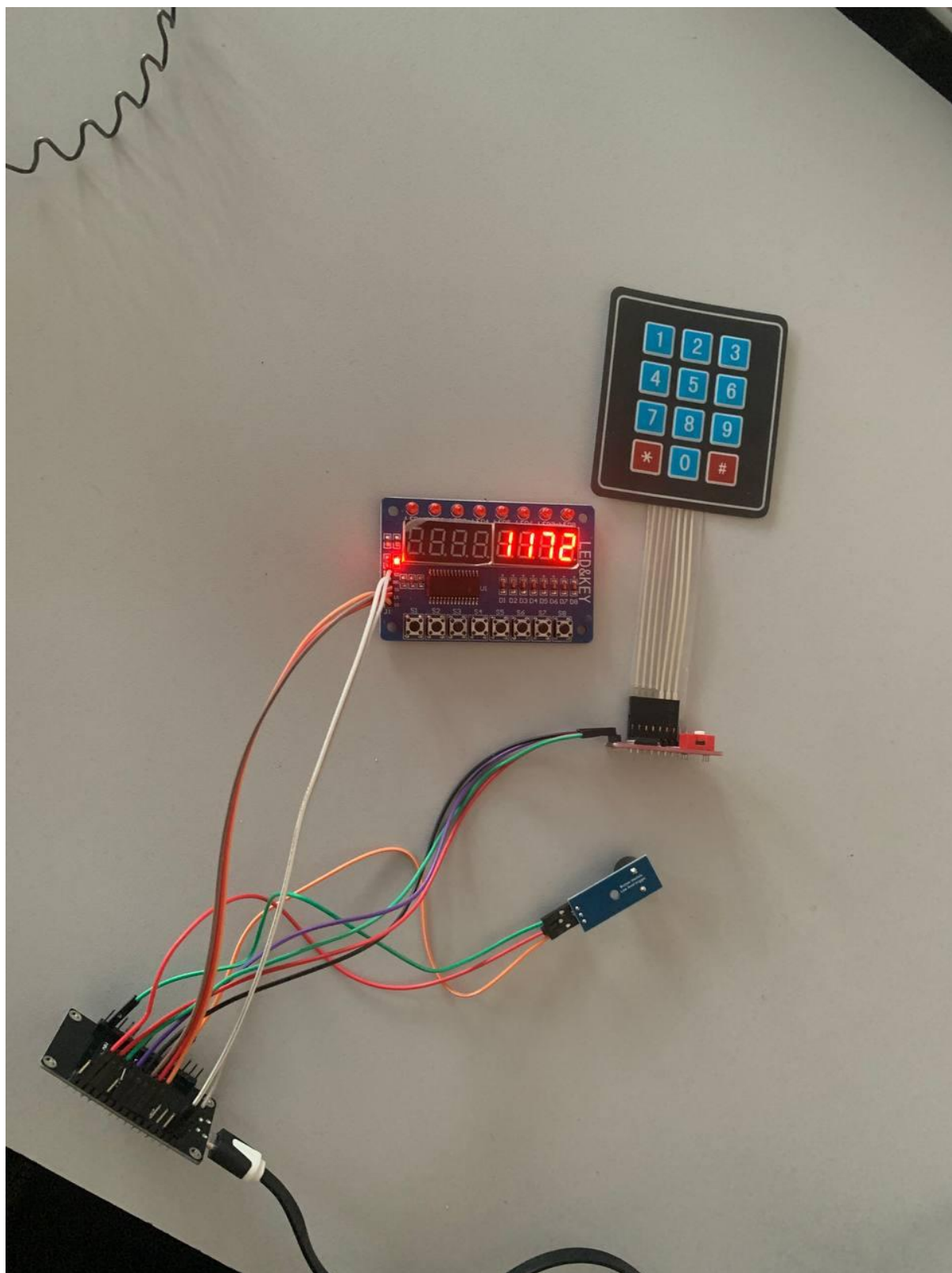












ARDODO REACTION TRAINER

Sign in

Username

Password

→ Login

No account yet? [Register](#)



Dashboard

Pairing

Devices

Leaderboard

Sessions

Logout

Dashboard

Current player, reaction stats, and live game state.

Player
nifreebie

Best result
430 ms

Average reaction
876 ms

Games
5

Round totals

Hits
38

Misses
3

Wrong
0

False starts
0

Current session

Status
ACTIVE

Device
trainer

Round
2 / 10

Best reaction
878 ms

Average reaction
878 ms

Counts
1 hit, 0 miss

Pairing

Generate a short code and enter it on the physical device.

+ Generate code

Pairing code

371259

Cancel code

Status

Active

Expires in

115s

Device

—

Devices

Paired trainers and their connection status.

Google Translate

Online devices

All devices currently visible to the backend.

1

trainer

nodemcu-02

Online

Firmware
1.0.0

Last seen
May 05, 08:50 PM

NAME	ID	FIRMWARE	STATUS	LAST SEEN	ACTIONS
trainer	nodemcu-01	1.0.0	Offline	May 05, 09:00 PM	Rename Unlink

Leaderboard

Fastest saved results across players.

Limit 20

PLACE	PLAYER	TIME	DEVICE	CREATED
#1	jija	343 ms	nodemcu-01	May 05, 08:02 PM
#2	nifreebie	430 ms	nodemcu-01	May 05, 08:48 PM

Sessions

Recent game sessions and summary reaction metrics.

STATUS	MODE	DEVICE	STARTED	ENDED	BEST	AVG	OPEN
FINISHED	CLASSIC	trainer	May 05, 08:59 PM	May 05, 08:59 PM	759 ms	1172 ms	Details >
FINISHED	CLASSIC	trainer	May 05, 08:48 PM	May 05, 08:48 PM	430 ms	689 ms	Details >
FINISHED	CLASSIC	trainer	May 05, 08:01 PM	May 05, 08:02 PM	616 ms	920 ms	Details >
ACTIVE	CLASSIC	trainer	May 05, 07:53 PM	—	878 ms	878 ms	Details >
FINISHED	CLASSIC	trainer	May 05, 07:51 PM	May 05, 07:52 PM	462 ms	771 ms	Details >

Session details

690ca507-76bb-4811-9814-63db5fc20029

← Back

Status

FINISHED

Mode

CLASSIC

Best reaction

759 ms

Average reaction

1172 ms

Summary

Device

trainer

Rounds

10 / 10

Started

May 05, 08:59 PM

Ended

May 05, 08:59 PM

Hits

8

Misses

2

Wrong

0

False starts

0

ROUND	TARGET	PRESSED	REACTION	RESULT	STATUS
1	6	6	1451 ms	HIT	COMPLETED
2	2	2	1315 ms	HIT	COMPLETED
3	1	—	—	MISS	COMPLETED
4	7	7	1221 ms	HIT	COMPLETED
5	6	—	—	MISS	COMPLETED
6	4	4	952 ms	HIT	COMPLETED
7	1	1	1286 ms	HIT	COMPLETED